

A stylized graphic of a human head in profile, composed of a blue wireframe mesh. The head is filled with glowing orange and yellow nodes connected by lines, representing a neural network or data flow. The background is a dark gray with a grid of white squares, some of which are highlighted in red.

Mapna AI

هکاتون هوش مصنوعی مپنا

Industrial Data Hackathon

مسابقه مقدماتی: ۷ شهریور

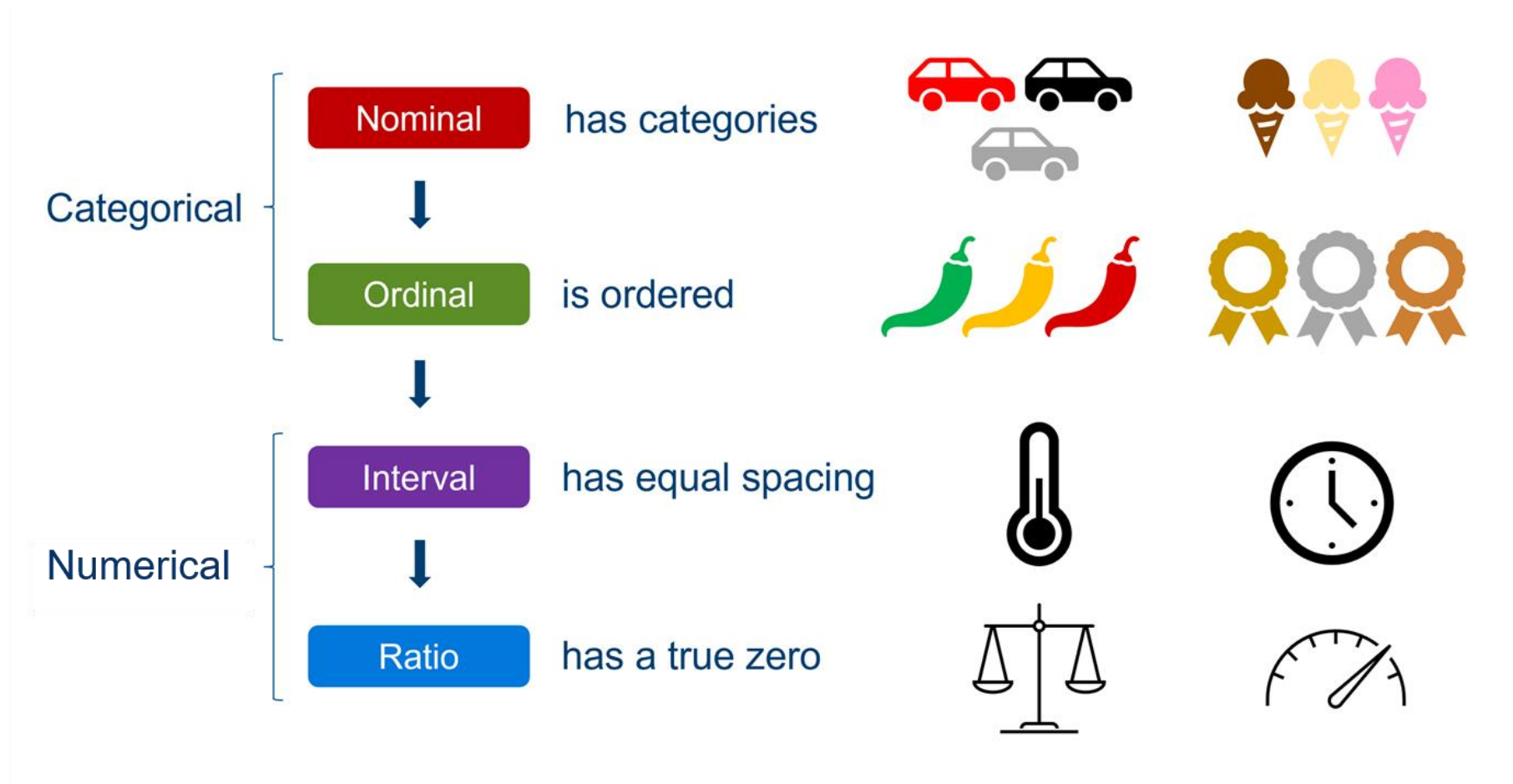


ML Process Steps

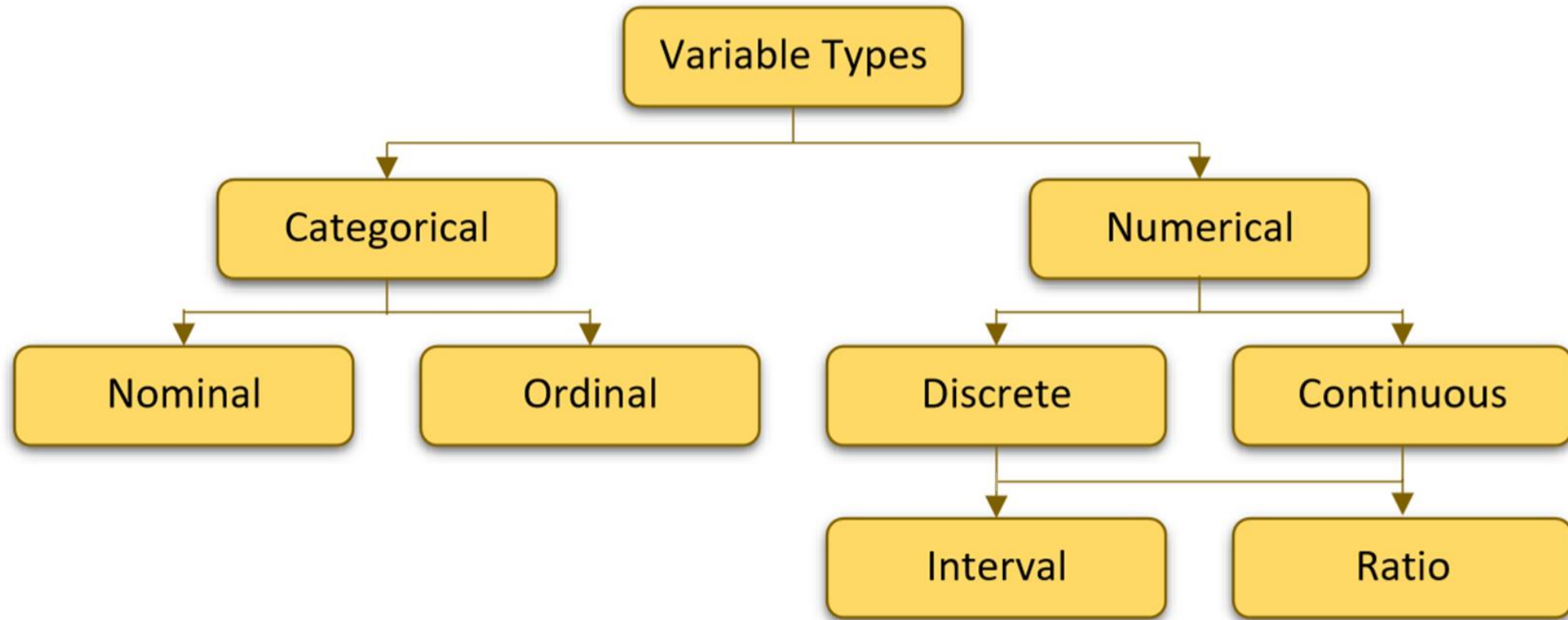
- Planning
- Data Preparation
- Model Engineering
- Model Evaluation
- Model Deployment
- Monitoring and Maintenance



Data Types

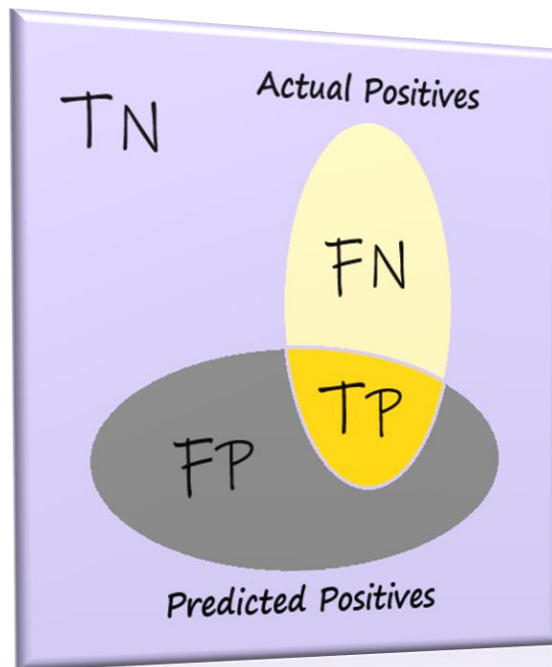


Data Types



Evaluation Metrics for Classification

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$



		Actual Values	
		Positive	Negative
Predicted Values	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

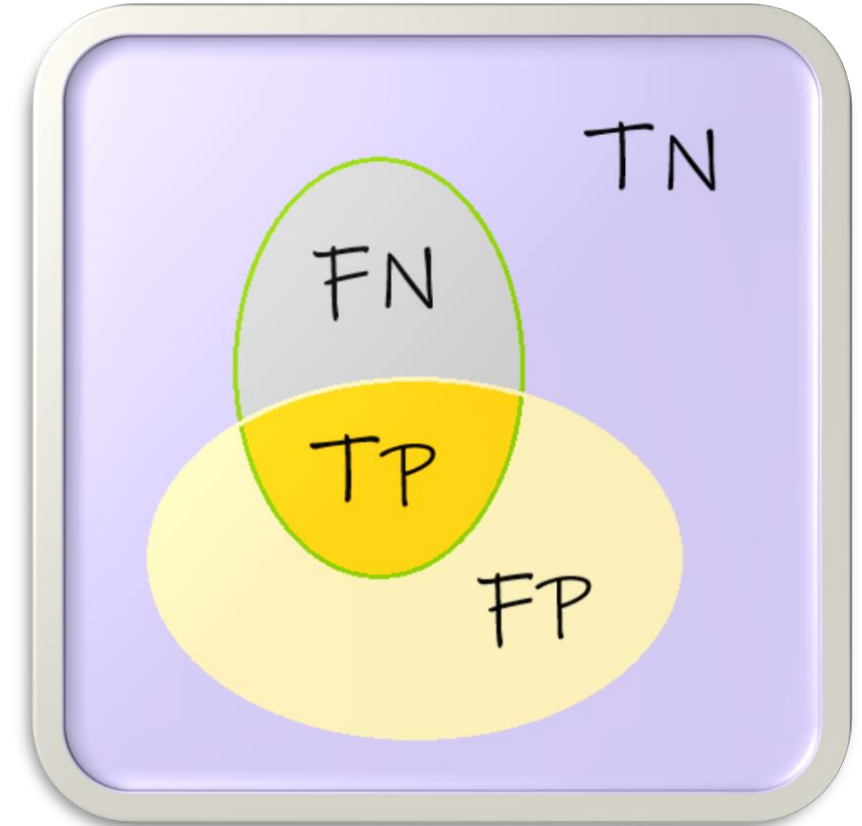
Evaluation Metrics for Classification

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

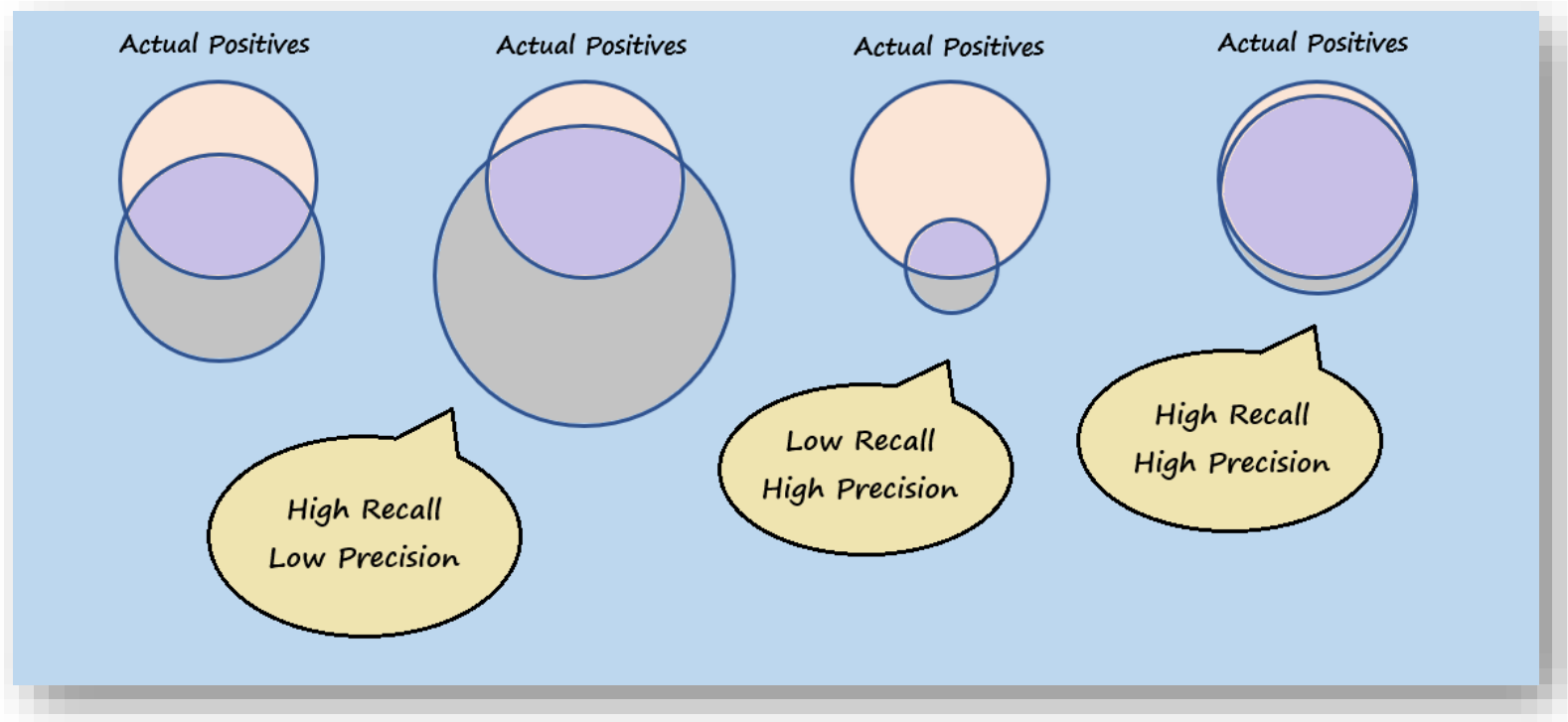
$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$F_{\beta} = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}$$



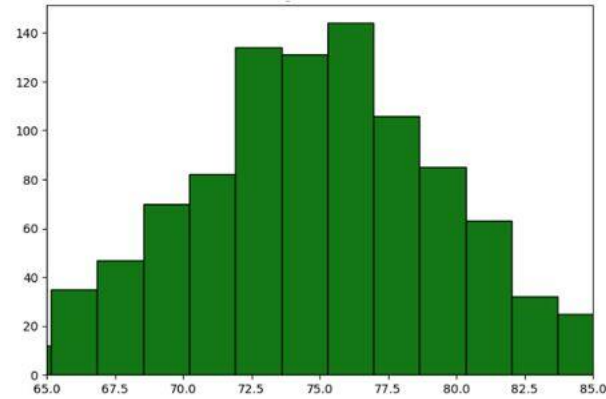
Precision vs. Recall vs. F1

Precision	Recall	F1
50	50	50
60	40	48
70	30	42
80	20	32
90	10	18

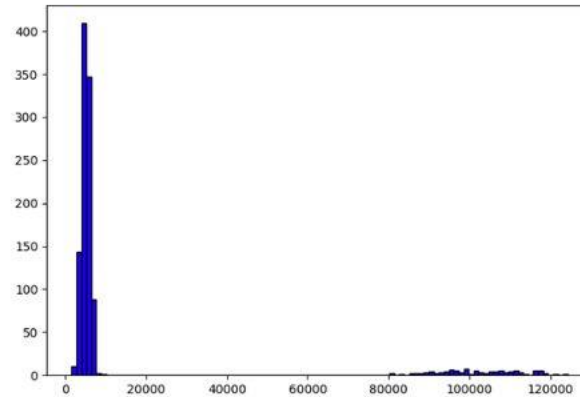


<https://encord.com/blog/classification-metrics-accuracy-precision-recall/>

Data Preparation

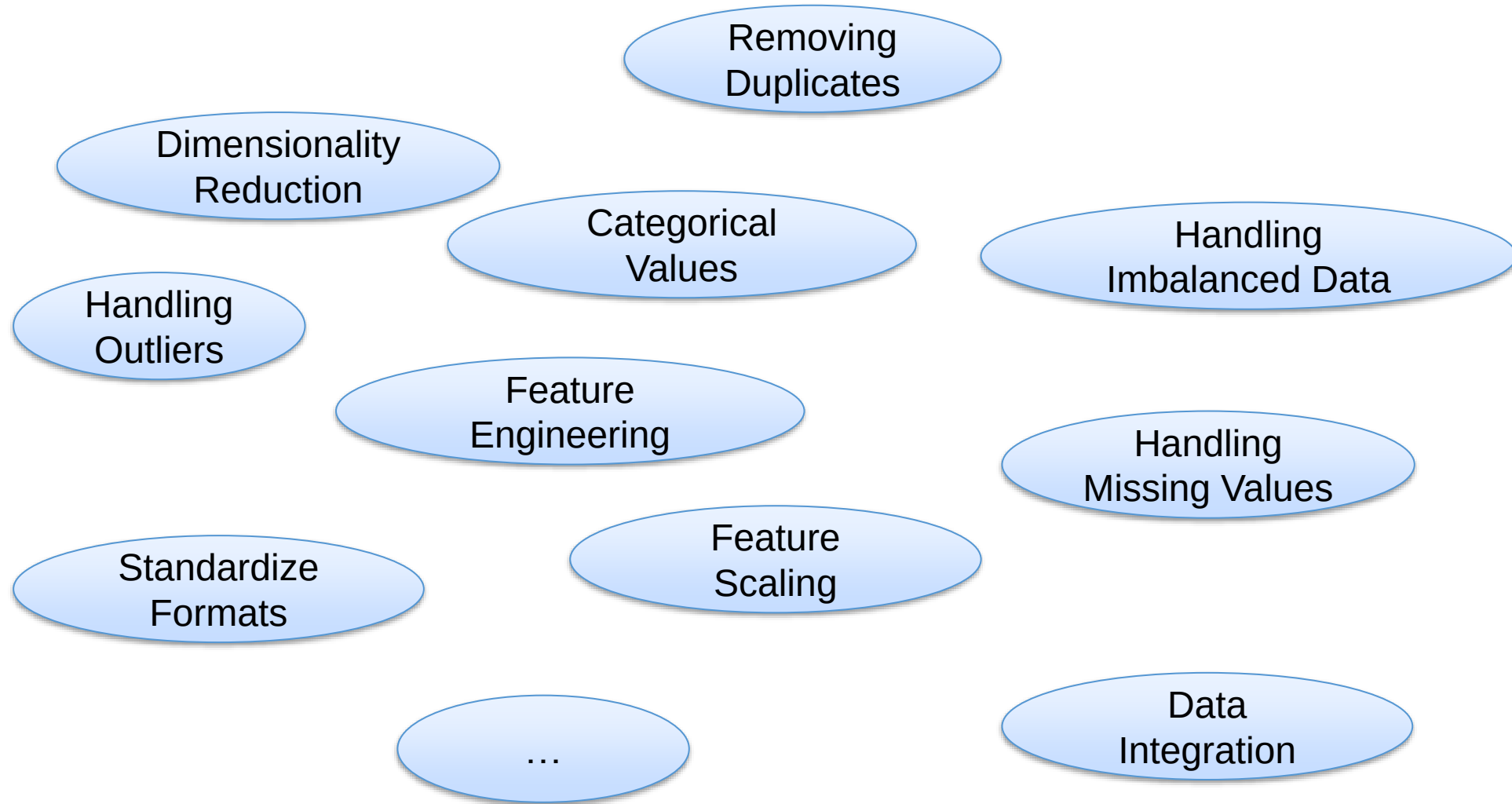


Histogram
in School



Histogram
in real life

Data Preparation



Handling Missing Values

- ✓ Rows Deletion
- ✓ Columns Deletion
- ✓ Mean/Median/Mode Imputation
- ✓ Regression
- ✓ KNN
- ✓ Interpolation
- ✓ Predictive Models
- ✓ ...

Handling Categorical Values

Label Encoding

Each unique category value is assigned an integer value

One-Hot Encoding

Each category value is converted into a new binary column, with 1 or 0

Ordinal Encoding

Similar to label encoding but specifically used when the categorical variable has an inherent order

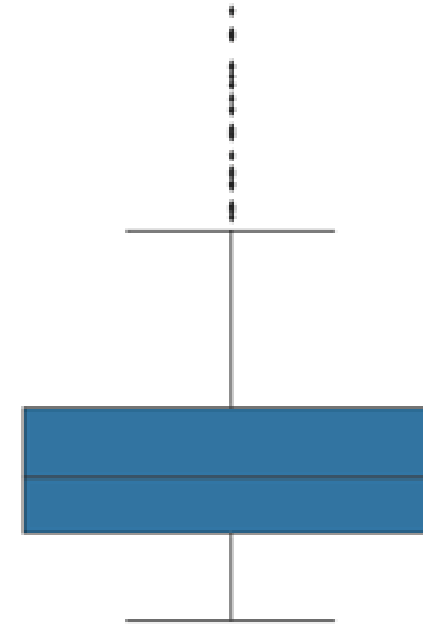
Binary Encoding

Converts categories into integers, then into binary, and splits the binary values into separate columns

Count Encoding, Mean Encoding, Dummy Encoding, Hashing, ...

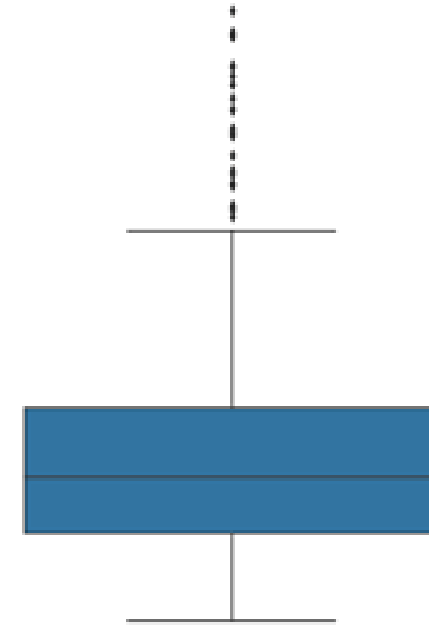
Outlier Detection

- ✓ IQR (Interquartile Range)
- ✓ Z-Score
- ✓ DBSCAN
- ✓ Isolation Forest
- ✓ One-Class SVM
- ✓ ...



Handling Outliers

- ✓ **Reporting**
- ✓ **Removing**
- ✓ **Imputation**
- ✓ **Transformation**
- ✓ **Capping and Flooring**
- ✓ **...**



Binning

Equal-width Binning

Divides the range of values into intervals of equal size

Equal-frequency Binning

Divides the values such that each bin has approximately the same number of data points.

Custom Binning



Feature Scaling

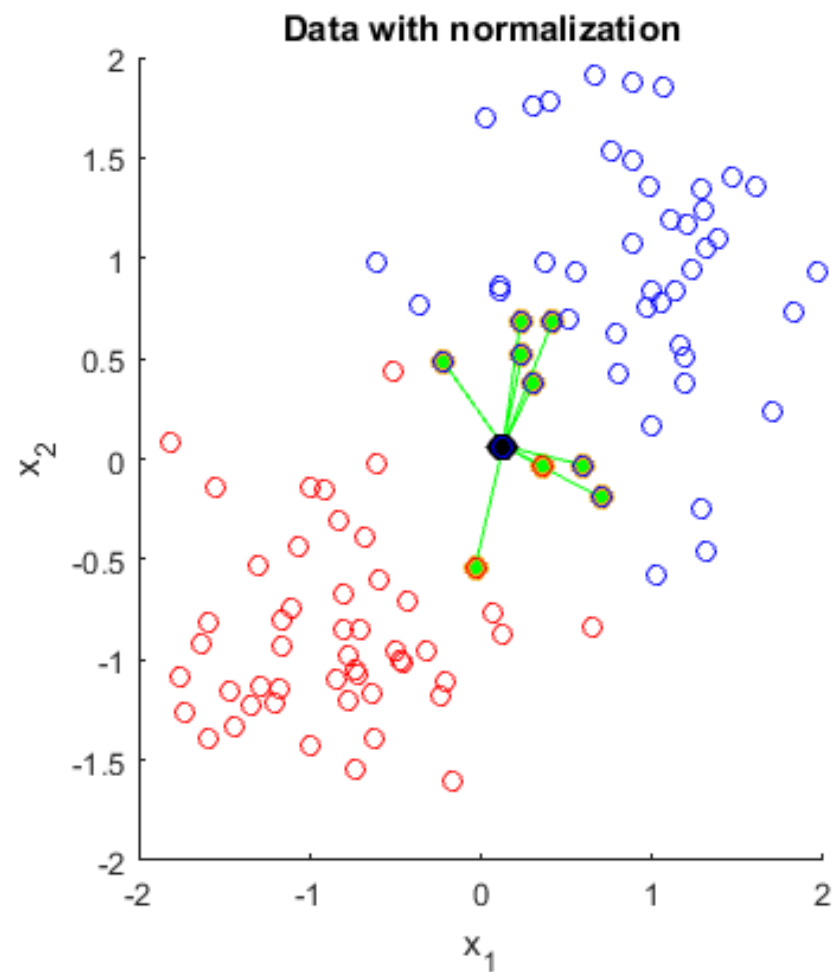
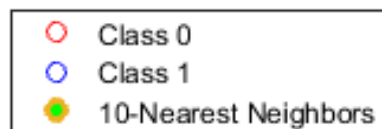
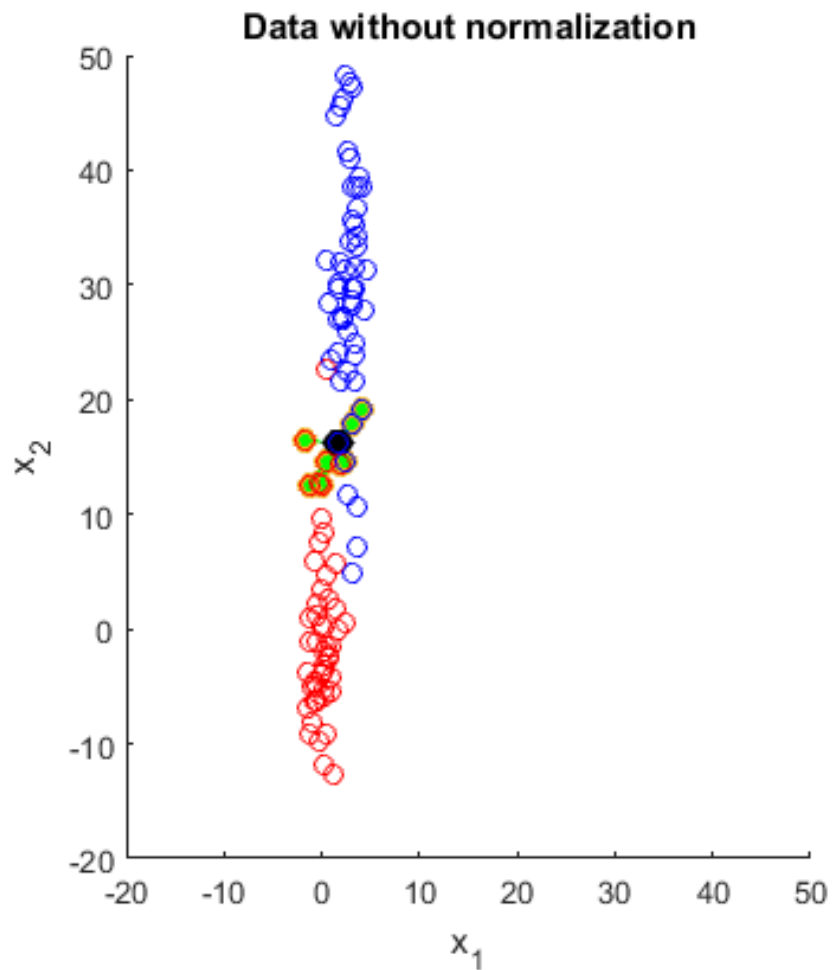
$$\text{Price} = w_1 \times \text{Area} + w_2 \times \text{Floor} + w_3 \times \text{Age} + b$$

$$Toman = Toman/m^2 \times m^2 + Toman/floor \times floor + Toman/year \times Year + Toman$$

$$X_{normal} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad X_{standard} = \frac{X - \bar{X}}{\sigma} \quad X_{robust} = \frac{X - X_{median}}{IQR}$$

$$Toman = Toman \times NoUnit + Toman \times NoUnit + Toman \times NoUnit + Toman$$

Feature Scaling



StandardScaler vs. MinMaxScaler vs. RobustScaler

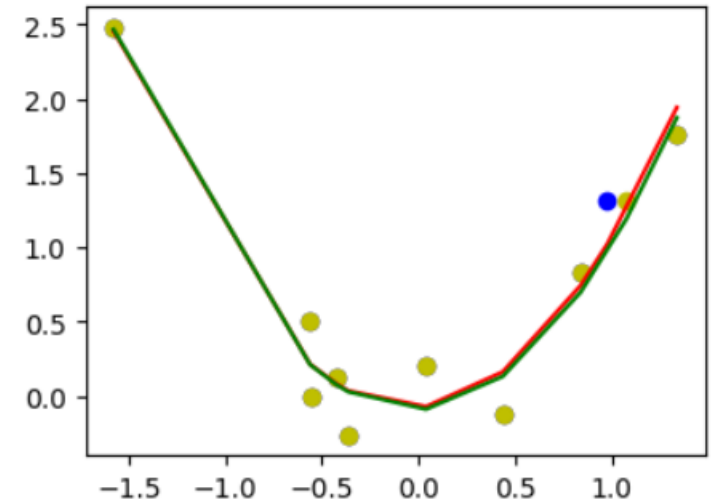
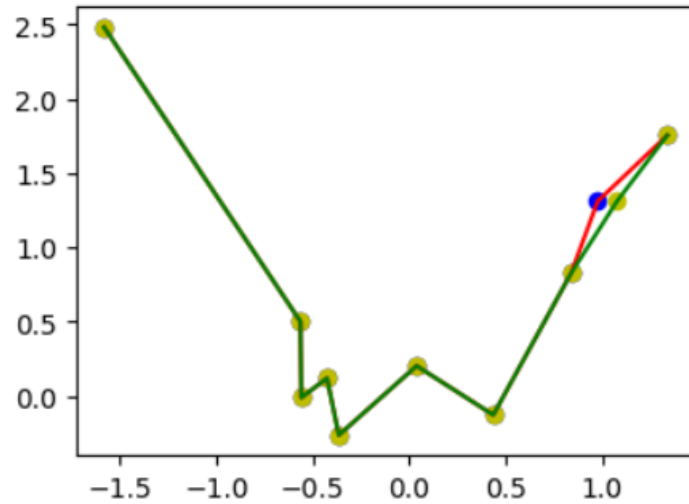
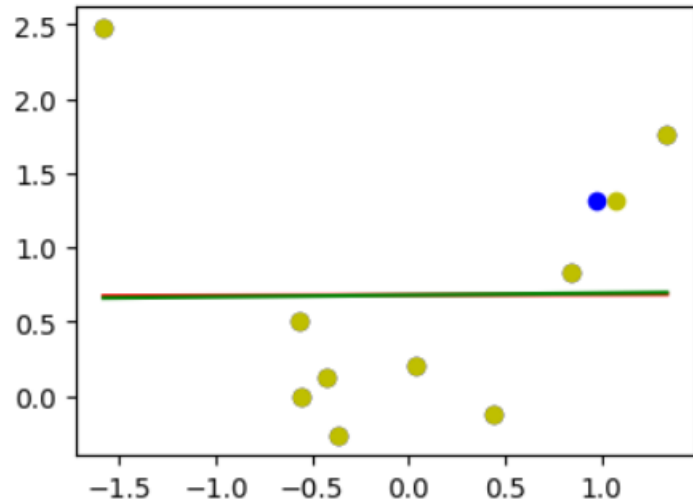
$$X_{standard} = \frac{X - \bar{X}}{\sigma}$$

$$X_{normal} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

$$X_{robust} = \frac{X - X_{median}}{IQR}$$

Aspect	StandardScaler	MinMaxScaler	RobustScaler
Based on	Mean and Standard Deviation	Minimum and Maximum	Median and IQR
Sensitive to outliers	Yes	Yes	No
Assumes normal distribution	Yes	No	No
Typical output range	Centered around 0	Typically [0, 1]	Centered around median
Best use case	Normally distributed data without significant outliers	Data without outliers; needed normalized range	Data with outliers

Overfitting & Underfitting



Overfitting & Underfitting – Model Complexity

Underfitting

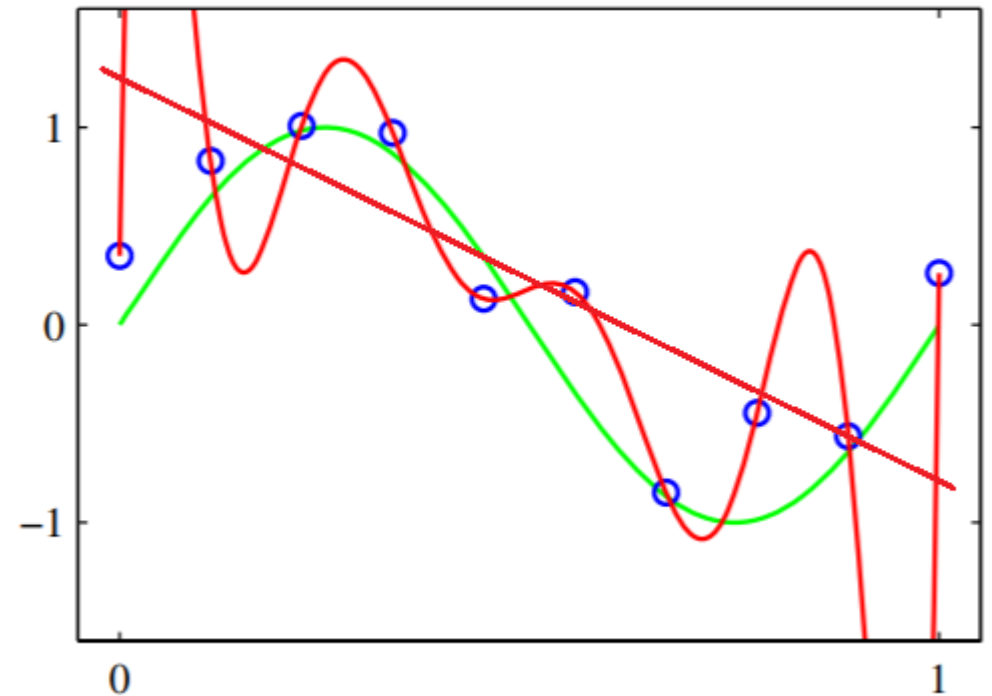
A model with too simple a hypothesis space, such as a linear model for a clearly non-linear problem, will not capture the underlying trend, leading to high bias.

Overfitting

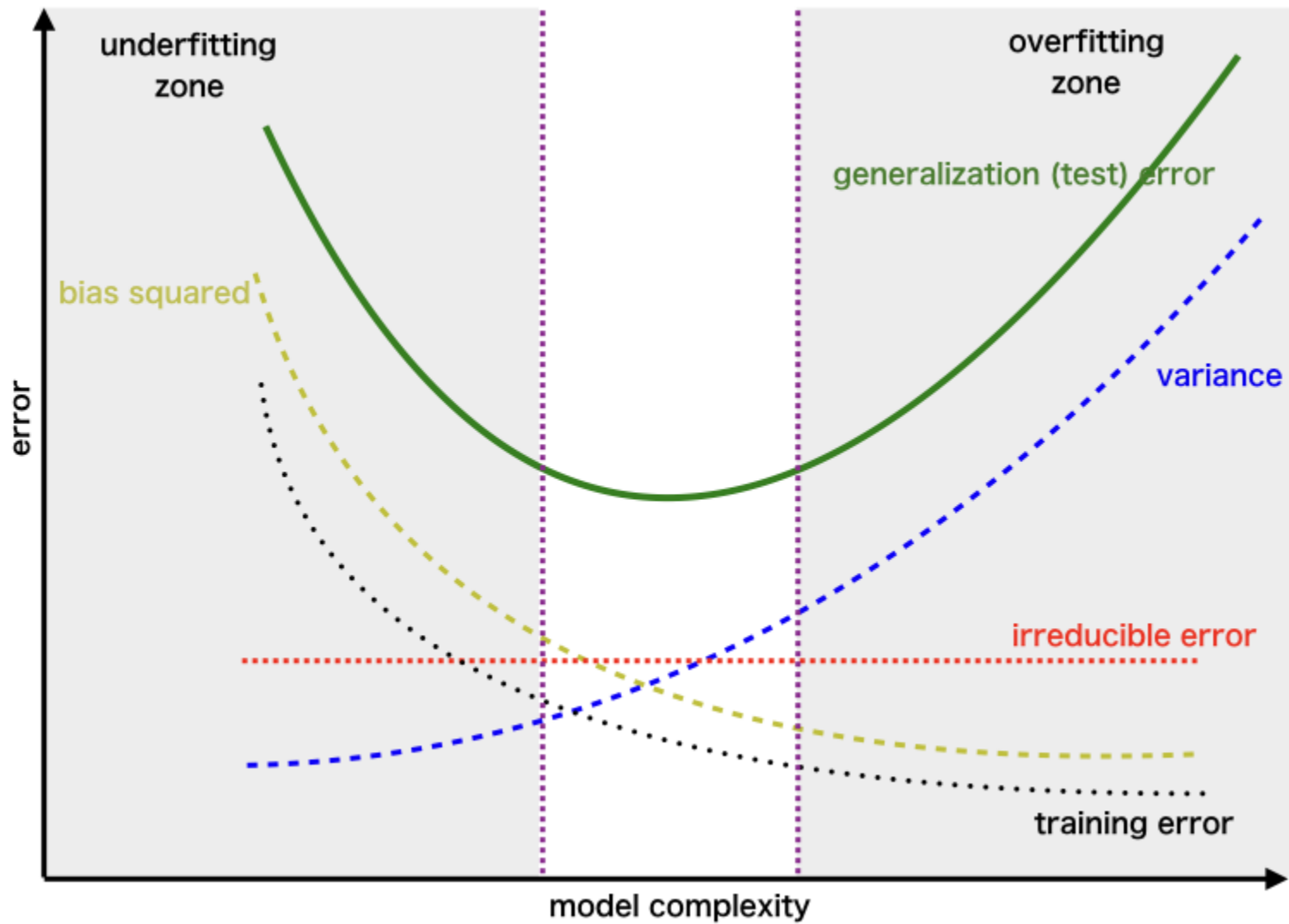
A model with a highly complex hypothesis space fits the training data very closely, capturing noise and fluctuations, leading to high variance and poor performance on new data.

Optimal Fit

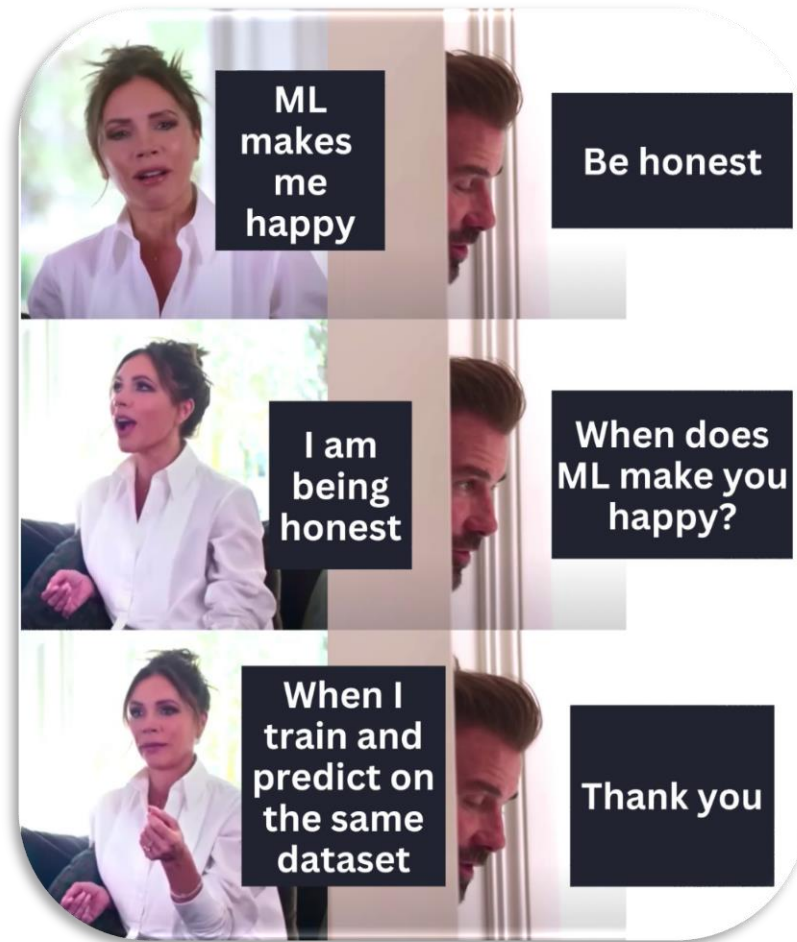
A model with an appropriately complex hypothesis space captures the underlying pattern without fitting the noise, leading to good generalization.



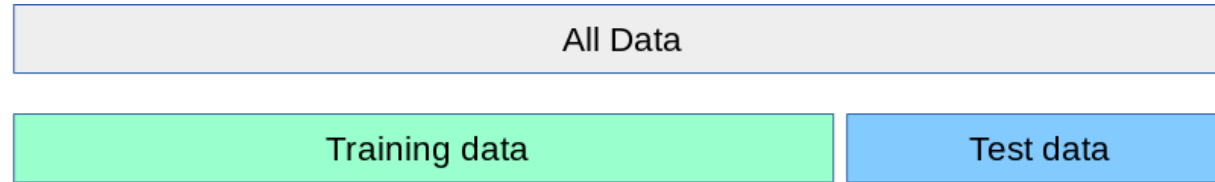
Bias & Variance Trade-off



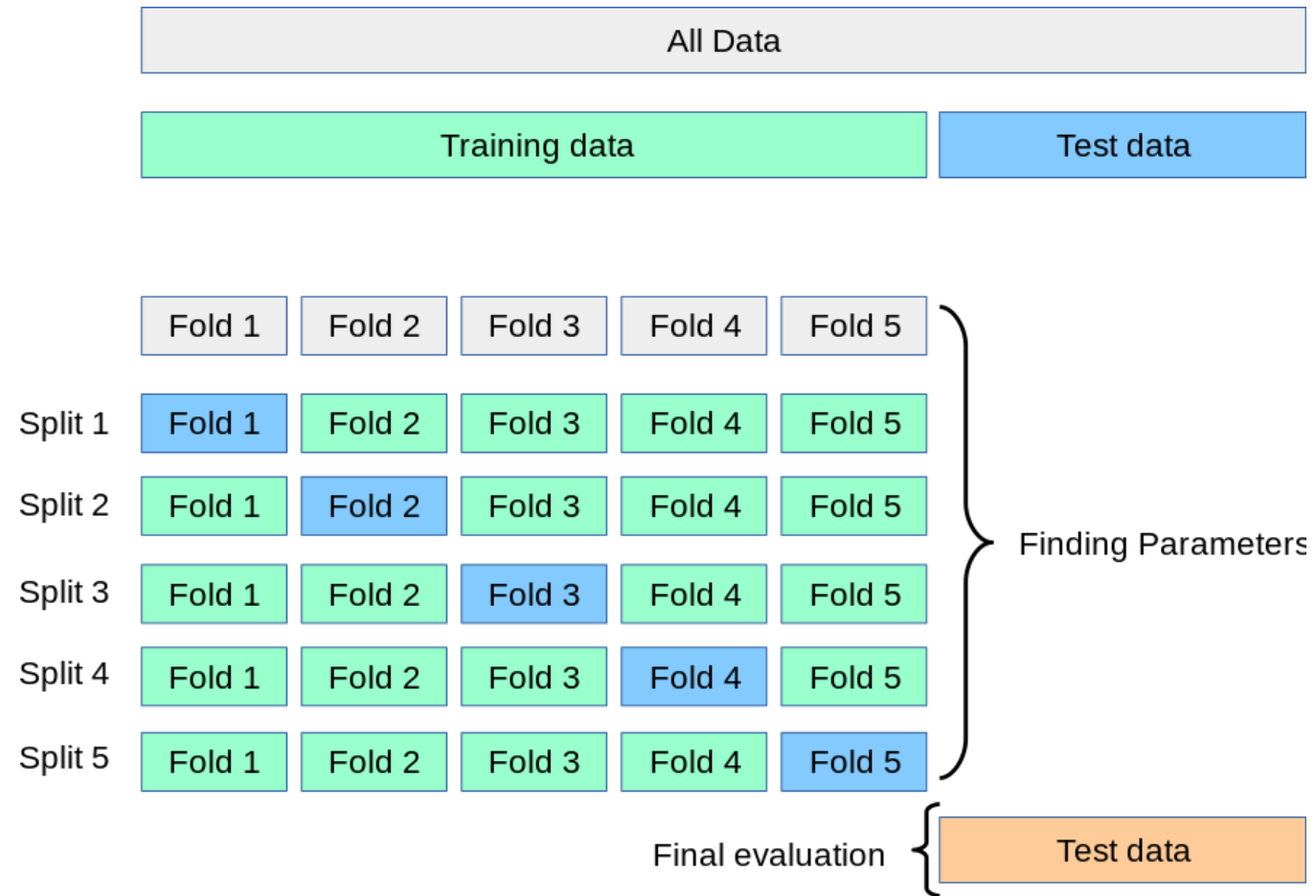
Train-Test Split



Train-Test Split



How to detect overfitting? K-Fold Cross Validation



Cross Validation

K-Fold

```
k_folds = KFold(n_splits = 5)
```

Stratified K-Fold

```
sk_folds = StratifiedKFold(n_splits = 5)
```

Leave-One-Out (LOO)

```
loo = LeaveOneOut()
```

Leave-P-Out (LPO)

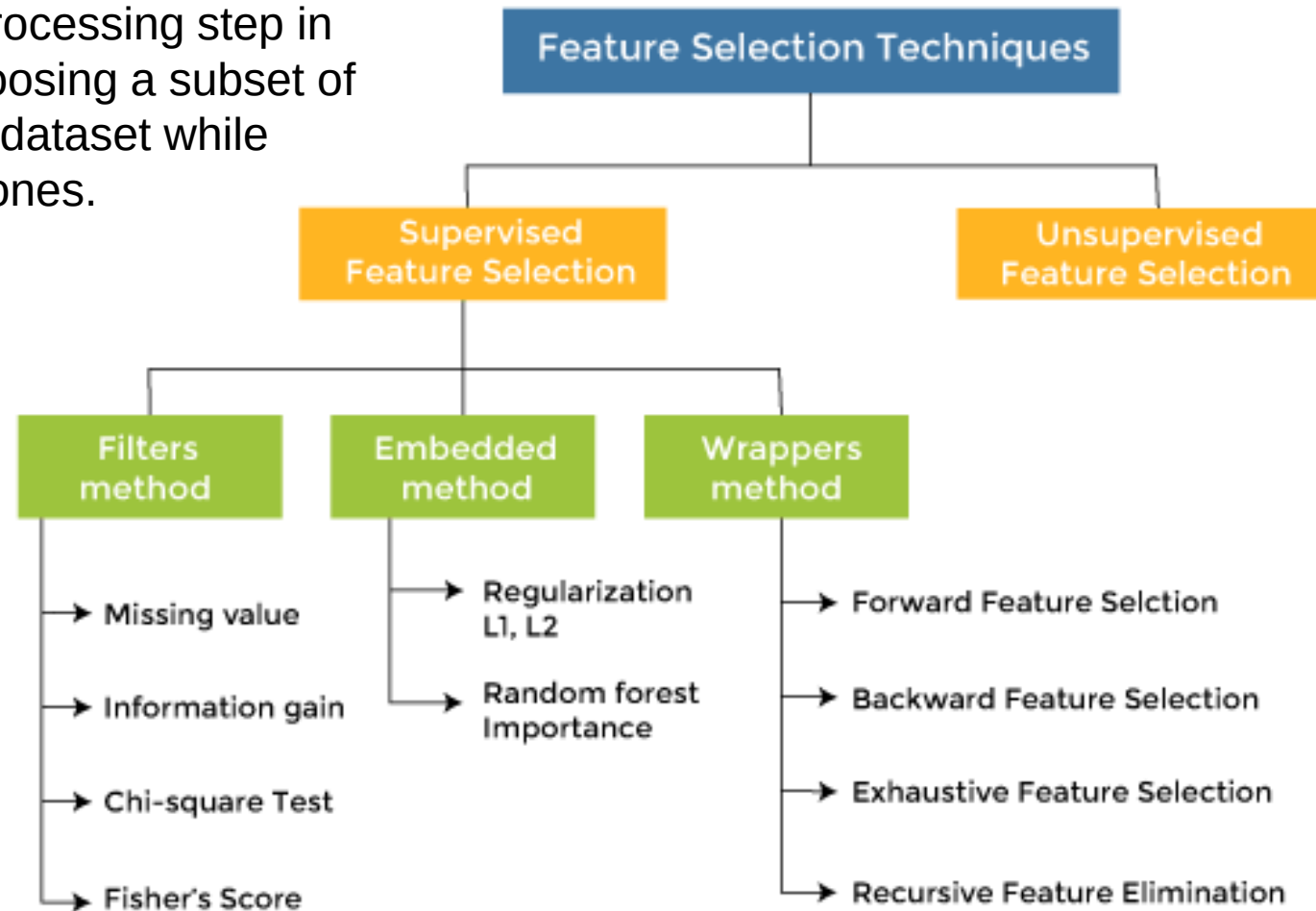
```
lpo = LeavePOut(p=2)
```

Shuffle Split

```
ss = ShuffleSplit(train_size=0.7, test_size=0.3, n_splits = 5)
```


Feature Selection

Feature selection is a critical preprocessing step in machine learning that involves choosing a subset of relevant features from the original dataset while excluding irrelevant or redundant ones.



Feature Selection – Why?

- Reduces the number of features, which can help combat the curse of dimensionality.
- Enhances the efficiency of the learning algorithm, reducing computation time and memory requirements.
- Focuses on the most informative features, leading to simpler and more interpretable models.
- Reduces the risk of overfitting, as models are less likely to capture noise or irrelevant patterns.
- Improves a model's ability to generalize to unseen data by reducing noise and improving its signal-to-noise ratio.
- Mitigates the risk of model bias, as fewer features can lead to more balanced datasets.
- Simplifies model interpretation and explanation by using fewer, more relevant features.
- Enables visualizations of data and model decisions in lower-dimensional spaces, aiding human understanding.
- Speeds up model training and testing, making it feasible to experiment with more algorithms or hyperparameters.

Feature Selection – Why?

- Benefits real-time or resource-constrained applications by reducing computation overhead.
- Helps identify and exclude features with missing values, outliers, or inconsistencies.
- Enhances the quality of input data, which can improve model performance.
- Eliminates noisy or irrelevant features that may introduce variance and hinder model accuracy.
- Focuses on the most informative aspects of the data.
- Facilitates the application of machine learning to large datasets, where managing high-dimensional data can be challenging.
- Makes it easier to apply machine learning techniques to big data problems.
- Guides feature engineering efforts by identifying which features are most relevant to the target variable.
- Helps domain experts and data scientists make informed decisions about feature creation or transformation.
- Reduces the risk of unintentionally introducing bias into models by selecting features that are ethically neutral.

Feature Selection – Filter Methods

Filter methods are a category of feature selection techniques that do not involve training a machine learning model but rely on statistical properties of the data.

- Variance Threshold

Variance thresholding removes features with low variance, as they are considered less informative and potentially redundant. Useful for eliminating constant or near-constant features.

- Feature Pairwise Correlation

Linear correlation measures the strength and direction of the linear relationship between two variables, typically using the Pearson correlation coefficient, which ranges from -1 (perfect negative correlation) to 1 (perfect positive correlation). Nonlinear correlation, on the other hand, accounts for more complex relationships that cannot be adequately described by a straight line, and various methods like Spearman's rank correlation or Kendall's tau are used to assess such relationships.

- Missing Value Ratio

Features with a high percentage of missing values may be less informative.

Feature Selection – Wrapper Methods

Wrapper methods are a category of feature selection techniques that assess the quality of a feature subset by training and evaluating a machine learning model using different subsets of features

- Forward Selection

It starts with an empty feature set and iteratively adds one feature at a time based on the model's performance until no improvement is observed. It's computationally efficient compared to exhaustive search and can work well for moderate-sized feature sets.

- Backward Elimination

It begins with all features and removes one feature at a time, evaluating the model's performance after each removal. It is less likely to overlook important features as it starts with the full set.

- Exhaustive Search

This method systematically evaluates all possible feature combinations and selects the best-performing subset. It guarantees finding the best feature subset but is computationally expensive for large feature sets.

Feature Selection – Wrapper Methods

- Recursive Feature Elimination (RFE)

RFE starts with all features and iteratively removes the least important features based on a ranking criterion (e.g., feature weights). It repeats this process until the desired number of features is reached.

It is efficient and effective, especially when combined with machine learning models that provide feature importance scores.

It helps prevent overfitting by iteratively removing less important features and can improve model interpretability.

RFE can help you automatically select the most important features for your model and is especially useful when you have a large number of features and want to reduce dimensionality while preserving predictive power.

Feature Selection – Embedded Methods

Embedded methods for feature selection involve using machine learning algorithms themselves to determine feature importance and select relevant features during the training process. Decision trees are a common choice for embedded feature selection.

Random Forest Feature Selection

A Random Forest is constructed by creating an ensemble of decision trees using bootstrapped samples of the dataset. It calculates a feature importance score for each feature in the dataset based on how much each feature contributes to reducing impurity or error in the tree nodes.

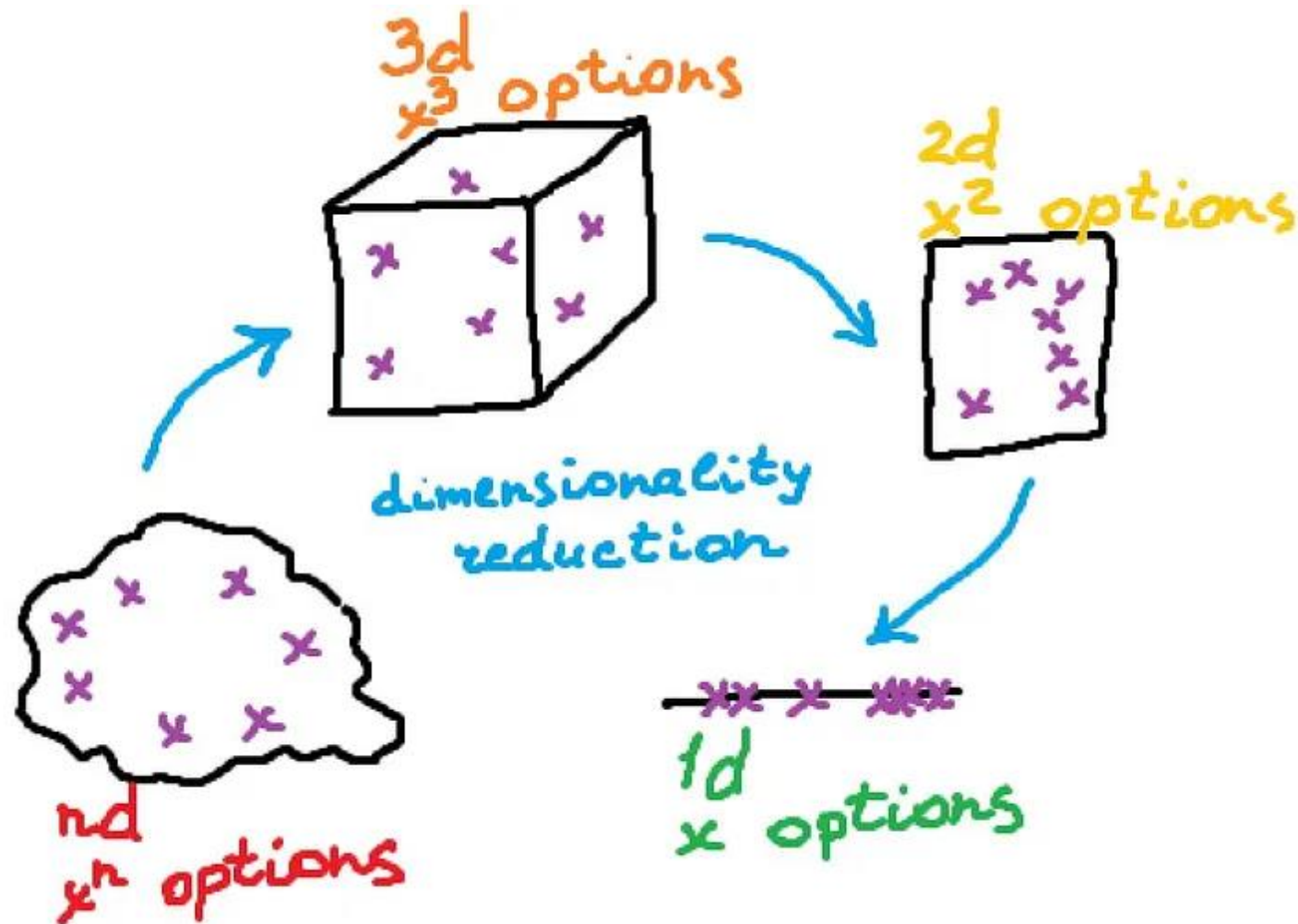
Once the Random Forest is trained, you can retrieve the feature importance scores. Then you can choose to select the top N ranked features based on their importance scores, or you can set a threshold for feature importance scores and select features that meet or exceed this threshold.

Feature Selection – Unsupervised Feature Selection

Unsupervised feature selection techniques are used to reduce the dimensionality of a dataset without using class labels or target variables. These methods focus on finding patterns, structures, or relationships among features in an unsupervised manner.

- Principal Component Analysis (PCA)
- t-SNE
- Independent Component Analysis (ICA)
- Non-Negative Matrix Factorization (NMF)
- Autoencoders
- Clustering for Feature Selection
- ...

Dimensionality Reduction

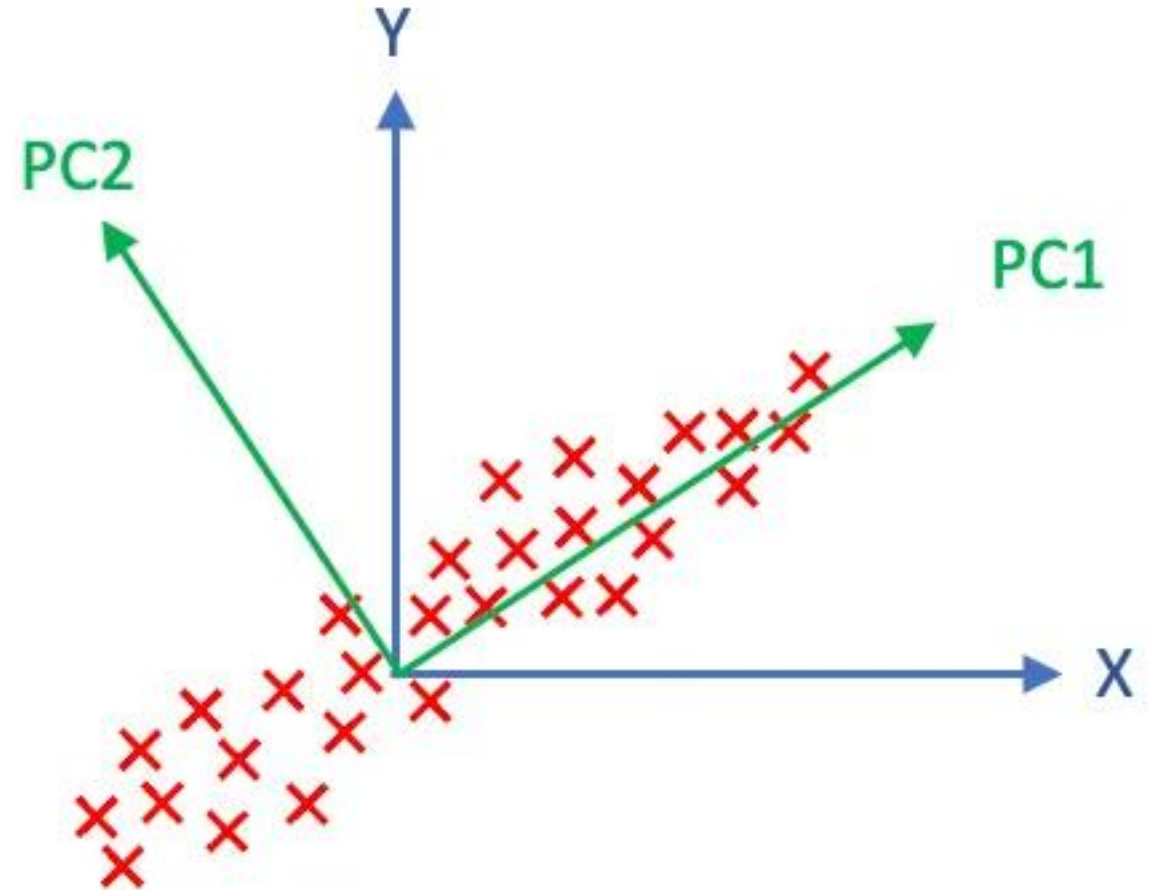


Principal Component Analysis (PCA)

STEPS

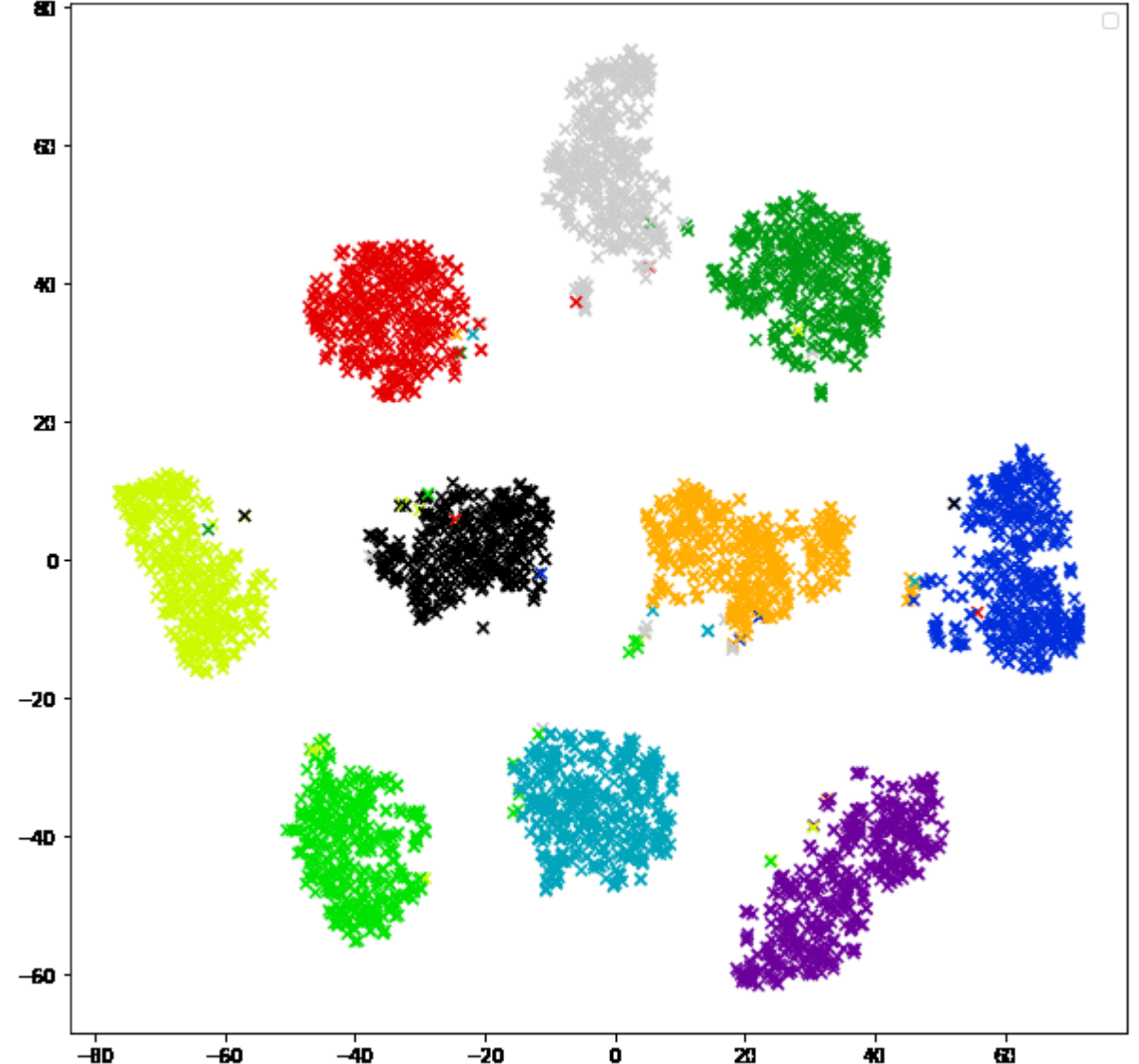
1. Data Standardization
2. Calculate Covariance Matrix
3. Compute Eigenvalues and Eigenvectors
4. Sort Eigenvalues
5. Select Principal Components
6. Projection & Dimensionality Reduction

$$\text{Cov}(x,y) = \frac{\sum (x_i - \bar{x}) * (y_i - \bar{y})}{N}$$



Dimensionality Reduction – t-SNE

- t-distributed Stochastic Neighbor Embedding, is a dimensionality reduction technique used primarily for visualization and exploration of high-dimensional data.
- In a t-SNE plot, data points that are close to each other in the plot tend to be similar or exhibit similar patterns in the original high-dimensional space.
- Denser regions suggest that data points in those areas are more similar to each other compared to points in sparser regions.
- Outliers or data points that are dissimilar from others in the high-dimensional space will appear as isolated points in the t-SNE plot, far from the main clusters or groups. These outliers might represent anomalies or unique data points.

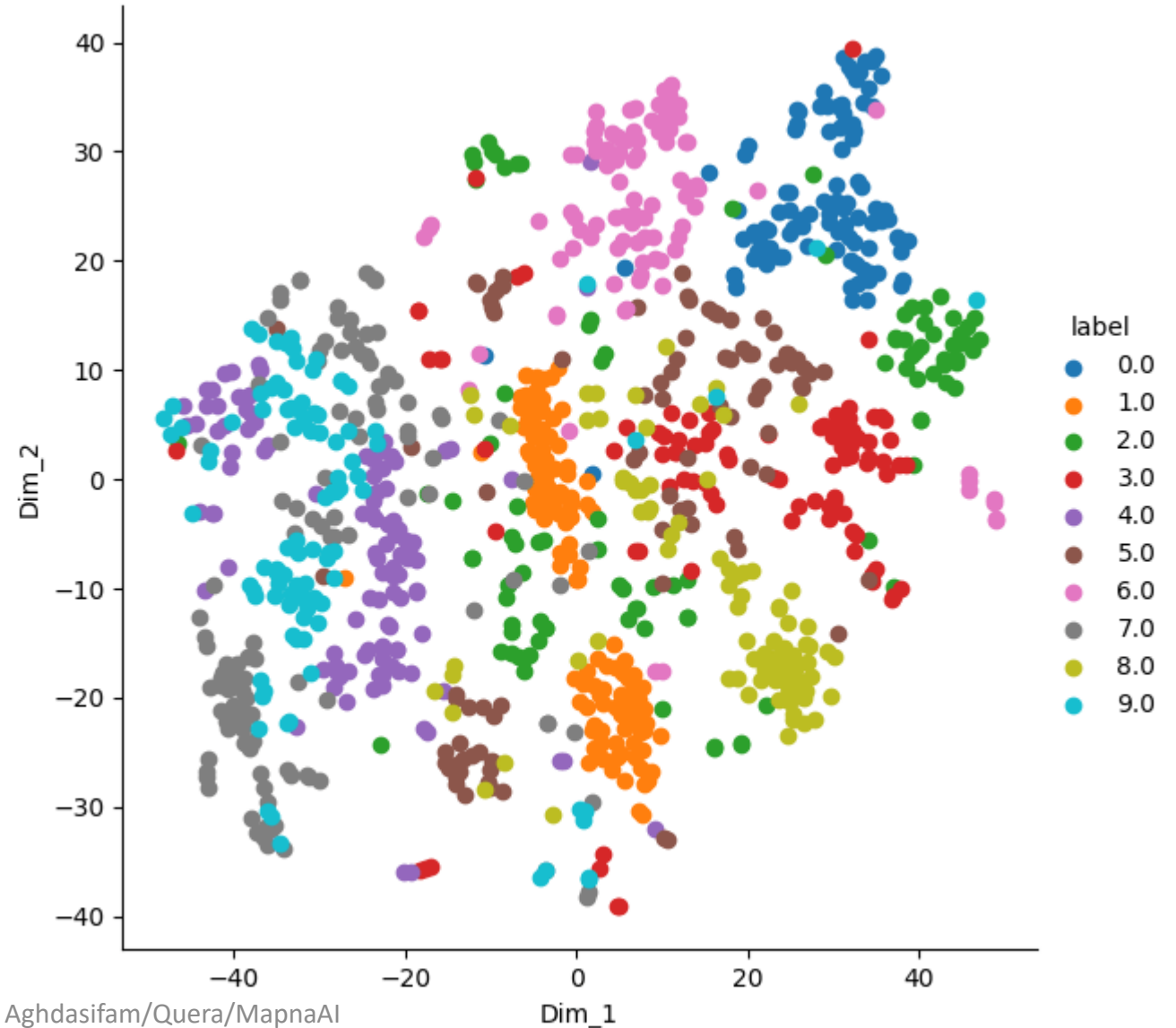


Dimensionality Reduction – t-SNE Algorithm

STEPS

- Initialize Low-Dimensional Embedding
- Calculate Pairwise Similarities
- Calculate Conditional Probabilities in Low Dimension
- Calculate Conditional Probabilities in High Dimension
- Minimize the Divergence
- Repeat the optimization process for multiple iterations, refining the positions of data points in the low-dimensional space.

enjoyalgorithms.com/blog/tsne-algorithm-in-ml



PCA vs t-SNE

- PCA is a linear dimensionality reduction technique focused on preserving global data structure, while t-SNE is non-linear, emphasizing the preservation of local similarities for visualization and clustering
- PCA is linear, meaning it finds linear combinations of original features, while t-SNE is non-linear, capturing complex relationships.
- PCA provides interpretable principal components with clear feature contributions, while t-SNE's lower-dimensional representation is less interpretable.
- PCA is computationally efficient and scalable, while t-SNE can be computationally expensive for large datasets due to pairwise computations.
- PCA is commonly used for feature engineering and dimensionality reduction, while t-SNE is mainly used for visualization, clustering validation, and exploratory data analysis.
- PCA is applied in various fields, including feature selection and data preprocessing, while t-SNE is frequently used in data visualization and understanding complex relationships in high-dimensional data.

Time-Series

Regression	Time Series
1. In Regression it does not matter if we reshuffle the data.	1. Time Series consists time from before to the latest and they cannot be shuffled.
2. In Regression data points are independent.	2. In Time Series there is strong correlation between successive values.
3. Regression model predicts only on the basis of the values given.	3. Time Series not only depends on the values given, but also on depends on the sequence in which the values are given.

So:

- Train-test split must respect time (no leakage from the future)
- Use time-aware CV (e.g., TimeSeriesSplit in scikit-learn)

An example

it`s
TIME FOR
#CODING

**THE
END**